

Implementation, Version Control, and Code Review

COMP-7431: Advanced Software Engineering | Week 9

CampusConnect: integrate RAG, voice, and avatar into one reviewable MVP

Build one vertical slice. Preserve the trail. Review before merge.

Monday Morning: Week 8 Validated the Pieces

Week 7 defined contracts and created voice. Week 8 tested the video experience. Week 9 must turn those validated decisions into one maintained vertical slice.

WHAT THE TEAM LEARNED

“Keep voice and video optional. Show sources. Put the transcript beside Play. Preserve a complete text path when media is skipped or fails.”

WEEK 9 IMPLEMENTATION QUESTION

Can the team connect RAG, voice, avatar video, fallbacks, and review evidence without hiding secrets, widening scope, or losing traceability?

A validated design becomes dependable only when its implementation is bounded, reviewable, and reversible.

Three Working Demos Are Not Yet One Product

SEPARATE PARTS

The local RAG app answers from campus IT documents; ElevenLabs speaks; HeyGen supplies a prerecorded introduction.

Example: Each tool works correctly when opened on its own.

INTEGRATION RISK

The parts may disagree on state, sources, errors, and what happens when a dependency is unavailable.

Example: Voice speaks an answer while the screen shows a different citation.

ENGINEERING OUTCOME

One user journey, one source of truth, explicit fallbacks, protected credentials, and one reviewable repository trail.

Example: The CampusConnect app preserves one grounded text answer and source; voice and video remain optional presentation paths around that same journey.

The Week 9 deliverable is not a collage of tools. It is one coherent vertical slice with understandable boundaries.

Week 9 Makes the MVP Reviewable and Traceable

IMPLEMENTATION DECISIONS

- Define one vertical slice and its component boundaries
- Keep secrets outside browser code and commits
- Integrate through explicit requests, responses, errors, and fallbacks

Example: The app records `answer_text`, `citation_url`, `voice_status`, and `video_state` rather than passing an unstructured mystery object between components.

COLLABORATION EVIDENCE

- Link the change to an issue and acceptance criteria
- Use focused branches and meaningful commits
- Complete peer review and revise before merge

Example: The pull request explains how muted use and text fallback were preserved.

The repository should explain what changed, why it changed, how it was checked, and who reviewed the result.

Implementation, Version Control, and Review Do Different Jobs

IMPLEMENTATION

Assemble behavior across real component boundaries.

EXAMPLE

Connect the approved RAG answer to optional speech while preserving the visible source.

VERSION CONTROL

Create a reversible, understandable history of the change.

EXAMPLE

A branch isolates multimodal integration; commits separate orchestration, fallback, and documentation.

CODE REVIEW

Ask another engineer to challenge the proposal before acceptance.

EXAMPLE

The reviewer finds that a failed voice request erases the user's typed question.

The code makes the journey work. The history explains it. Review tests whether it deserves to join the accepted product.

Work in a Sequence a Reviewer Can Follow

1 ISSUE

Name the user-visible change

Example: Integrate grounded text, optional voice, presenter video, and text fallback.

2 BRANCH

Isolate one vertical slice

Example: feature/91-multimodal-journey starts from current main.

3 COMMITS

Record coherent steps

Example: define shared state; connect voice; add video + transcript; preserve text fallback.

4 PULL REQUEST

Present evidence for review

Example: link criteria, screenshots, test questions, and limitations.

A clean trail reduces reviewer detective work and makes correction safer.

The AI Periodic Table Places Week 9 at Integration

INTERACTION LAYER —> COLUMN VI

Prompts, system instructions, guardrails, and voice shape the user exchange.

The user may speak, type, skip, replay, or continue silently.

SYSTEM INTEGRATION —> COLUMN VII

APIs, functions, SDKs, and structured state connect software components.

CampusConnect aligns RAG, voice, video, citations, and fallbacks.

AGENTIC DESIGN —> COLUMN VIII

Goals, tool boundaries, and guardrails keep optional AI paths subordinate to the user's task.

The system chooses text, speech, abstention, or fallback without creating a second truth.

AUTOMATION —> COLUMN X (WEEK 12)

Week 12 will add n8n only after the MVP is tested and red-teamed.

GitHub now preserves code, contracts, decisions, evidence, and peer review.

The architect is working chiefly across Interaction and System Integration—not training another model.

Scope One Vertical Slice Before Opening Tools

VERTICAL-SLICE STATEMENT

“A student can open CampusConnect, view or skip the presenter, ask one campus IT question by text or voice, receive a grounded answer with a source, and continue in text when media fails.”

- One supported password-reset question and one unsupported question
- Existing RAG remains the source of answer and citation
- Voice follows the approved CampusConnect knowledge and fallback contract
- Avatar remains a prerecorded orientation, not a live decision-maker
- The interface preserves state when either media path is skipped or fails
- GitHub records code, contracts, evidence, and review

A vertical slice crosses the necessary layers, but it does not include every feature in the roadmap.

Translate Validation Evidence into Implementation Criteria

JOURNEY CRITERION

State the complete user-visible outcome.

Example: A student can skip video, ask by text, inspect the source, and finish without voice.

BOUNDARY CRITERION

State which component owns each responsibility.

Example: RAG owns answer evidence; the app owns journey state; ElevenLabs owns voice; HeyGen remains prerecorded.

FAILURE CRITERION

State what meaning and control survive a failure.

Example: If voice times out, retain the answer and focus "Continue in text."

Acceptance criteria protect the validated behavior when implementation details change.

Trace One Complete Multimodal Journey



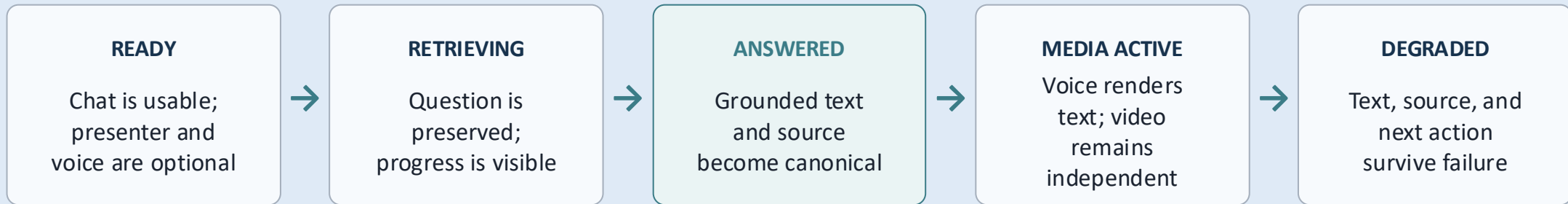
USER-JOURNEY PROMPT

“Find out how to reset a campus password. Use voice if you wish, inspect the source, and continue even if the presenter or speech path is unavailable.”

Do not instruct the user to click every control in order. The integrated design should reveal the optional paths.

The vertical slice succeeds only when the user goal survives every optional-media choice.

Integration Requires Explicit States and Ownership



WHY OWNERSHIP MATTERS

Every state needs one responsible component, a structured payload, a user-visible signal, and a fallback.

Example: The voice handler returns `status=voice_unavailable`; the interface retains `answer_text` and `citation_url`.

If two components both decide the answer, drift is inevitable and review becomes guesswork.

Write a Vertical-Slice Contract Before Integration

REAL IN THIS MVP

- Real CampusConnect RAG answer and citation
- Real optional ElevenLabs voice path
- Real prerecorded HeyGen video and transcript
- Real interface state, error, and text-fallback behavior

DEFERRED OR SIMULATED

- Production identity, scaling, analytics, and billing
- Live conversational avatar behavior
- Every campus office and policy source
- n8n workflow automation and escalation—introduced in Week 12

Integration is disciplined exclusion: build enough to prove the slice without pretending the deferred system already exists.

The Issue, Branch, and Definition of Done Must Agree

ISSUE

Describe the validated need, scope, acceptance criteria, and non-goals.

Example: Issue #91 integrates one grounded IT journey with optional media and text fallback.

BRANCH

Name the isolated line of work after the issue and outcome.

Example: feature/91-multimodal-journey—not update-stuff-final.

DEFINITION OF DONE

Name the evidence required before review and merge.

Example: supported + unsupported questions, failure path, no secrets, updated diagram, peer approval.

Traceability is strongest when the issue, branch, commits, pull request, and final behavior tell the same story.

Choose the Branch Pattern by the Change

Change pattern	Recommended branch	CampusConnect example
Vertical slice	feature/<issue>-<outcome>	feature/91-multimodal-journey
Focused repair	fix/<issue>-<defect>	fix/94-preserve-text-fallback
Documentation	docs/<issue>-<artifact>	docs/95-integration-contract
Unrelated discovery	Open a new issue and branch	Do not mix a dashboard redesign into Issue #91

The branch should isolate one reviewable outcome. A separate branch is not a separate permanent product.

Commits Tell the Integration Story

COMMIT SEQUENCE EXAMPLE

- 1. Define multimodal state and ownership contract*
- 2. Preserve RAG answer + source as the canonical result*
- 3. Connect optional voice and degraded text state*
- 4. Embed presenter video + transcript and update evidence*
- 5. Repair reviewer finding: preserve question after timeout*

WHAT

One coherent change

WHY

The user or system
consequence

SCOPE

Only files required for this
step

VERIFY

Evidence the step still works

A readable commit history lets a reviewer understand progression, isolate a regression, and see how feedback changed the code.

Secrets Belong Behind the Interface Boundary

BROWSER + REPOSITORY

Treat browser code and committed files as public.

Example: Never place an ElevenLabs key or other private service token in JavaScript, screenshots, or GitHub source.

PROTECTED SERVICE BOUNDARY

Use the provider's hosted agent or store credentials in a server-side environment—not in code sent to the browser.

Example: The page opens a hosted ElevenLabs agent, or a protected server function calls the private API.

TEAM CONFIGURATION

Commit names and setup guidance—not secret values.

Example: .env.example contains ELEVENLABS_API_KEY= with no value; .env remains ignored.

Deleting a visible key from the latest file is not enough. Revoke or rotate it because Git history and logs may still contain it.

Integrate by Contract—not by Tool Guesswork

REQUEST INSIDE THE APPLICATION

- Preserve the typed question and session identifier
- Validate required fields before retrieval
- Carry mode=text or mode=voice explicitly
- Avoid collecting unnecessary personal data

Example: { question, session_id, mode } reaches the CampusConnect handler; no service key appears in the browser.

RESPONSE TO THE INTERFACE

- Return answer_text, citation_url, status, and fallback
- Return voice_url only when generation succeeds
- Keep the same schema for success and degraded states
- Never let an error become an empty screen

Example: status=voice_unavailable still returns answer_text and citation_url.

The contract keeps CampusConnect portable even if a tool or provider changes.

Give Every Component One Clear Responsibility

EXPERIENCE

The existing Gradio UI renders chat, disclosure, source, media controls, and fallback.

EXAMPLE

A skipped video never disables the question box.

APPLICATION INTEGRATION

CampusConnect keeps one shared state for the question, grounded answer, citation, media, and fallback.

EXAMPLE

A voice failure routes to the same approved text answer.

SPECIALIZED SERVICES

RAG supplies grounded content; ElevenLabs renders optional speech; HeyGen supplies the prerecorded presenter.

EXAMPLE

The avatar introduces CampusConnect but never answers the live question.

Clear ownership makes failures diagnosable and code review meaningful.

Design Integration Failures Before They Surprise the Demo

Failure	Integrated behavior	Evidence for review
No RAG support	Abstain and route to the correct campus office	Unsupported question produces no invented citation
Voice unavailable	Show grounded text and source; offer retry	Typed question and answer remain visible
Video blocked	Show Play or transcript; keep chat active	Muted and keyboard-only journey still completes
Service timeout	Return controlled status and text next step	No empty screen, duplicate request, or exposed detail

A vertical slice is incomplete if it can demonstrate success but cannot explain what happens when a dependency fails.

Authors and Reviewers Have Different Responsibilities

AUTHOR PREPARES THE PROPOSAL

- Link issue and acceptance criteria
- Keep the diff focused
- Explain architecture and secret handling
- Provide evidence for success and failure paths
- Name known limitations

REVIEWER CHALLENGES THE PROPOSAL

- Compare behavior with the issue
- Read the diff and commit sequence
- Inspect boundaries, secrets, and fallbacks
- Reproduce the riskiest journey
- Approve, comment, or request changes

Weak review: “Looks good.” Strong review: “Voice failure retains the answer, but the source link loses keyboard focus; please preserve the next action.”

Peer review is not politeness or fault-finding. It is a shared decision about whether the change belongs in main.

A Review Checklist Exposes Integration Risk

BEHAVIOR

Does the complete supported and unsupported journey meet the issue?

BOUNDARIES

Does one component own each answer, state, and fallback?

SECURITY

Are secrets absent from browser code, commits, screenshots, and logs?

RESILIENCE

Do voice, video, retrieval, and service failures preserve text and next action?

TRACEABILITY

Can the reviewer follow issue → branch → commits → diff → evidence?

EXAMPLE REVIEW FINDING

“The voice path exposes a private key in browser code. Use the hosted agent or move the call behind a protected server function, then rotate the exposed key.”

Review the riskiest boundary first; do not spend the entire session correcting formatting.

Merge Only When the Latest Proposal Is Defensible

Merge question	Required evidence	CampusConnect example
Does the slice meet scope?	Issue and acceptance criteria are satisfied	One supported + unsupported journey works
Were risks addressed?	Secrets, fallbacks, and known limitations reviewed	No browser key; text survives media failure
Was feedback resolved?	Comments answered; changes pushed and re-reviewed	Reviewer verifies corrected source focus
Is the trail complete?	Focused commits and current approval	PR links issue, evidence, and latest commit

Approval is evidence that the latest diff was examined against shared criteria—not merely that the author completed the task.

Review Feedback Must Change the Artifact or the Rationale

KEEP

One grounded answer and one visible citation source

Review confirmed the answer and displayed source remain aligned.

CHANGE

Move private voice call behind a hosted or server-side boundary

Browser inspection exposed the request credential.

REMOVE

Second LLM call that rewrites the answer for speech

It could drift from the grounded text and citation.

ADD

Controlled voice-timeout response and retained question

Failure test produced an empty state.

GitHub trail: review comment → author reply → focused repair commit → updated evidence → re-review → resolved conversation.

A review is complete when the concern is addressed and the latest artifact supports the response.

Week 9 Integrates Existing Assets—Without n8n Yet

EXPERIENCE

The existing Gradio interface presents chat, source, avatar, controls, disclosure, and fallback.

The interface contract remains portable if the builder changes.

AI + MEDIA SERVICES

CampusConnect RAG grounds the answer; ElevenLabs adds optional voice; HeyGen supplies prerecorded video.

No tool silently becomes a second source of truth.

GITHUB

Version the code, contracts, evidence, commits, pull request, and peer-review revision.

The repository tells the engineering story another student can reproduce.

Week 12 will add n8n workflow design after Week 10 testing and Week 11 Red Teaming.

Week 9 Builds the MVP; Week 10 Tests It

“A working integration becomes engineering when another person can understand it, challenge it, reproduce it, and safely change it.”

- Start from validated behavior
- Scope one vertical slice
- Assign clear component ownership
- Protect credentials behind server-side boundaries
- Preserve issues, branches, and coherent commits
- Open a focused pull request
- Respond to peer evidence with revision
- Merge only the latest reviewed proposal

NEXT MONDAY — WEEK 10

Software Testing, Quality Assurance, and Continuous Integration

Separate deterministic software tests from probabilistic AI evaluation; set CI gates, defect priorities, and a release standard for the integrated CampusConnect MVP.